OCL

OCL: UML'in resmi bir parçası

Şimdiye kadar hep diyagramlara baktık.
- Ama diyagramlar herşeyi anlatamaz.
- Tanımlarda, tasarımda daha detay gerektiren noktalar vardır --> OCL

OCL bir belirtim (spesifikasyon) dilidir. Sistem özelliklerinin işlevsel detaylarını tanımlamanızı sağlar.
- Bildiren (declarative: Yapılacak kodu ve adımları açıkça belirtmeden sonuçları tanımlar)
- Güçlü tipli (Strongly typed: tüm değişkenler belli ve bir tipi diğeri yerine kullanamazsınız)
- Kısıtlar, koleksiyon sınıfları, diyagramlardaki sınıflar/ilişkiler arası gezinim mekanizması

Pek çok araç OCL'i destekler
- Rational Rose, ArgoUML, Eclipse, Poseidon/Octopus, Enterprise Architect, …

OCL'e neden ihtiyaç duyarız?

Diyagramlarla yapısal ilişkileri ve davranış tanımlarını çok iyi tanımlayabiliriz.

Kesin anlamsal detayları tanımlayacak bir mekanizma diyagramlarda yoktur.
- Anlamsal: Bu bileşenin bu görevi yapması ne demektir?

OCL, UML'i şunlarla genişletir:
- Değişmeyen sınıf yapısı
- İşlemlerin ön ve son koşullarının kesin tanımları
- Durum diyagramlarında geçişlerin kontrolü

OCL: Genel Bakış

Declarative (bildiren): Prosedürel değil, programlama dili değil.

Tamamen ifade dili. Aktiviteleri değil, değerleri tanımlar.

Atama ifadesi yok. = simgesi ile bildiri, kısıt, özellik ifade edilir.

Strongly-typed (güçlü tipli): Kendi veri tipleri ve UML diyagramlarındaki veri tipleri vardır.

Sadece bir anahtar konsept vardır: Kısıt (constraint) = Sistem özelliklerinin biçimsel bildirimi.

Yapabilecekleriniz:
- Sınıflarda değişmeyecekleri tanımlayabilirsiniz.
- İşlemlerde ön ve son koşulları tanımlayabilisiniz.
- Türetilen özellikler için türetme kurallarını tanımlayabilirsiniz.
- Durum diyagramlarında geçişler için kontrol (guard) tanımlayabilirsiniz.
- Mesajlar için hedef tanımlayabilirsiniz.
- Stereotipler için tip sabitleri (type invariant) tanımlayabilirsiniz.
- Bir sorgu dili olarak, sınıf diyagramında türetilen nesneleri belirli kriterlere göre sorgulayabilirsiniz.

OCL Sözdizimi

Dilde sadece tek bir ifade vardır: Kısıt

```
context <identifier> <constraintType>: <Boolean expression>
```

Kısıtta anahtar kelimeler ve ifadeler yer alır.

- İlk anahtar kelime: «context» Context, diyagramda nerede olduğumuzu gösterir.
- Sonra tanımlayıcı <identifier>: context'e ismini verir. Genellikle bir sınıfın adıdır. Belirli bir metodun adı da olabilir.
- İkinci anahtar kelime: Kısıtın tipi: (Invariance, Precondition, Postcondition)
- Sonra mantıksal ifade: İfadenin anlattığı kısıtın ne olduğu.

Kısıtlar UML sınıf modeli ile birebir ilişkilidir. Modeli oluşturmuş olmamız ve detay bilgileri OCL ile vermemiz beklenir.

OCL Değişmezleri (Invariant)

Bir özelliğin her zaman doğru olan ifadesi.

Sistemin anahtar bir gereksiniminin tanımı olarak düşünebiliriz.

- Ör. Sistemdeki nesnelerin değerleri arasında gerekli bir ilişki.

Anahtar kelime: «inv»

```
context BüyükŞirket
inv: çalışanSayısı > 50
```

- BüyükŞirket sınıfında çalışanSayısı özelliği 50'den büyük olmalıdır.

Veritabanındaki veri bütünlüğü kısıtlarına (integrity constraints) benzer.

- İşçi şirkette çalışır. --> Şirket iflas etti, veritabanından silindi. İşçi nerede çalışır??
- Programlamadaki avare işaretçi (dangling pointer) durumu oluştu. Veritabanı yönetim sistemi bu durumu engeller.
- OCL değişmez kısıtları da sistem tanımlamalarında aynı rolü üstlenir.

Ön ve Son Koşullar

UML'de işlemlerin anlamını tanımlar. Bir sınıfa ait işlemi kullanmak ne anlama geliyor?

pre: İşlemin anlamlı bir şekilde çalışması için ne doğru olmalı?
- Belirli bir işlemin gerçekleşmesine izin verilen koşulları belirtir

post: İşlem tamamlandıktan sonra ne garanti ediliyor?
- Belirli bir işlemin çalışmasının sonuçlarını belirtir.
  - Dönüş değeri ile girdi parametreleri arasındaki ilişkiyi tanımlar.
  - İşlemin tetiklenmesi ile sınıfta oluşacak özellik değişimlerini tanımlar.

Bir kısıtta sadece birini ya da ikisini birden tanımlayabiliriz.

Ayrı kısıtların birinde pre, birinde post da tanımlayabiliriz.

Ön ve Son Koşullar: Karekök

Ön koşul: Karekök rutininin argümanı negatif sayı olmamalıdır.

Son koşul: Hesaplanan sonucun karesi, argümana eşit olmalıdır.

Biraz tersine düşünme gibi olsa da, eşitliğin doğru ifadesidir. Kareköknün böyle davranması gerekir.

OCL:

```
context Real::squareRoot() : Real
pre: self >= 0
post: self = result * result
      and result >= 0
```

Özellik Değerlerinde Değişiklik

Karekök örneğinde bir fonksiyonun hesaplanması ile sonuçların özellikleri tanımlanmıştı.

Belirli bir işlemin etkisi olarak özellik değerlerinde değişiklik de olabilir. Son koşullar ile bunu da tanımlayabiliriz.

Örnek: Banka Hesabı. Bakiye özelliği var. Para yatırma ve çekme işlemleri var. Para yattığında bakiye güncellenmeli.

```
context Account::deposit(Real : amount)
pre: amount > 0
post: balance = balance + amount
```

Dikkat: OCL bildirimsel bir dildir. Buradaki = işareti atama değil, eşitliktir!! Ifade yanlış olur. Değer değiştiriyorsak, @pre kullanmalıyız. → işlem çalışmadan önceki değer.

```
post: balance = balance@pre + amount
```

Özellik Değerlerinde Değişiklik

a ve b isimli Real tipinde iki özelliği olan bir sınıfımız olsun.

Sınıfta swap isimli bir işlem var, a ve b'nin değerlerini takas ediyor.

Son koşul ile nasıl ifade ederiz?

post : _____ = _____ and _____ = _____

post : a = b@pre and b = a@pre

Bir programlama dilinde ihtiyacımız olan geçici değişkene burada gerek yok ☺

OCL Mevcut Basit Veri Tipleri

Tip	Değerler	İşlemler
Boolean	True, false	and, or, xor, not, implies, if-then-else
Integer	1, -5, 2, 34, 26524, …	*, +, -, /, abs(), …
Real	1.2, 3.14, -45.678, …	*, +, -, /, floor(), …
String	'merhaba dünya'	toUpper(), concat()

OCL Anahtar Kelimeleri

OCL dilinde küçük bir anahtar kelime kümesi vardır	Anahtar Kelime	Tanım
Bunları gördük	`inv, pre, post`	Kısıtları tanımlar
Koşullu ifadeler için	`if-then-else-endif`	Koşul ifadesi
Mantıksal operatörler	`not, or, and, xor, implies`	Mantıksal operatörler
UML'in birimleri bölütleme yeteneğini gösteren paketleme	`package, endpackage`	Paketler
Bunu da gördük	`context`	Kısıtları tanımlar
Tanım yapmamızı sağlayan çeşitli kelimeler	`def`	Global tanım
Bunlarla kısa hallerini kullanabiliriz	`let, in`	Lokal tanım
hesaplanan özellik değerini tanımlayabiliriz	`derive`	Özellik türetme
ilk değer atamasını init ile yapabiliriz	`init`	İlk değer tanımı
Bunu da gördük	`result`	İşlemin sonuç değeri
Bunu da gördük	`self`	Kısıtlanan nesnenin kendisi

"Let" İfadesi

Yerel kısaltma/tanım yapabilmemizi sağlar.

Kısıtlarınızın birinde, birden fazla defa kullanacağınız karmaşık bir hesaplama olsun.
- Her seferinde tekrar yazabiliriz ama hata olasılığı yüksektir.

Let ifadesi ile ifadenin değerini taşıyan yeni bir tanımlayıcı türetebilir ve bunu kısıtlarımızda kullanabiliriz.

Ör: income, işlerimizdeki tüm maaşlarımızın toplamı olsun.

```
let income: Integer = self.job.salary->sum() in
     if isUnemployed then income < 100
                     else income >= 100
endif
```

income ifadesini koşulun içinde tekrar tekrar yazabilirdik.

Gezinti

En başta ne demiştik?

"Kısıtlar, koleksiyon sınıfları, diyagramlardaki sınıflar/ilişkiler arası gezinim mekanizması"

OCL'in tipik olarak belirli bir sınıf diyagramı ile ilişkili olduğunu söylemiştik.
- Her bir kısıt ifadesi belirli bir sınıf veya işlem ile başlıyordu.

Kısıtlarımızda sadece context sınıfını değil, ilişkili başka sınıfların da özelliklerini değiştirebilmeliyiz.
- Bir OCL değişmezinde non-context sınıfın özelliklerine referans vermek için nitelikli isim kullanmalıyız

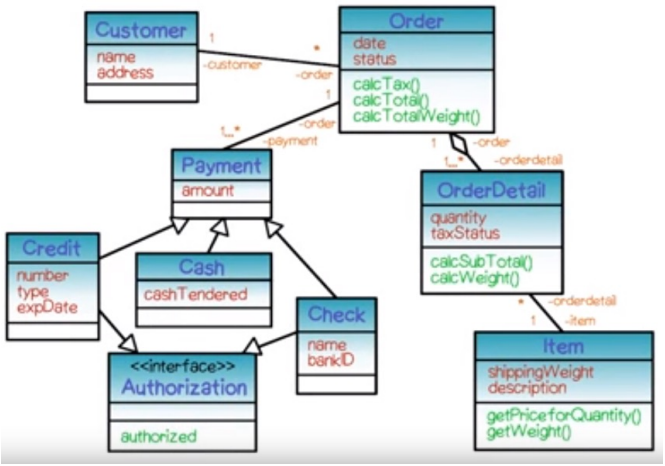
OCL'deki Gezinti (navigation) mekanizması ile diyagramda yürüyebiliriz ve her adımda, nokta ile ilişkideki takip eden sınıfı ya da ilişkiyi ekleyebiliriz.

Gezinti Örneği

Customer sınıfı bizim contextimiz olsun.

Bir siparişin hangi tarihte verildiği ile ilgili bir kısıt yazalım.

self.order.date ile erişebiliriz.



Gezinti Çoğullama

Sınıf diyagramlarında sayı ve * ile ilişkilerin çoğullamalarını (bir sınıftan kaç örnek diğer sınıftan kaç örnek ile ilişkiye geçecek) verebiliyorduk.

1 – 1: gelin/damat 1 – n: anne/çocuk m – n: öğrenci/ders

UML'de çoğullama kullanıldığında, gezintinin sonucu tekil bir nesne değil, bir koleksiyondur. Küme, yığın, sıralı dizi olabilir.

OCL'de bu koleksiyonlar üzerinde işlem yapabiliriz.

-> sembolü ile koleksiyonun adını işlemin adından ayırırız.

Önceki örnekte ödeme birden fazla bankanın çeki ile yapılsın. Sayı nasıl bulunur? (koleksiyonda size() işlemi vardır)

Customer nesnesindeyiz:

```
self.Order.Check.bankID->size()
```

Koleksiyonlar

Kısıtlara ve gezintiye baktık. OCL'deki üçüncü ana elemanı koleksiyon sınıfları.

OCL'de 1-n ve n-m tipinde ilişkileri destekleyen dört çeşit koleksiyon mevcuttur:

Koleksiyon (Collection)
- Küme (set)
- Yığın (bag)
- Sıralı dizi (sequence)

Koleksiyon sınıfının işlemleri:
- Size, includes, count, includesAll, isEmpty, notEmpty, sum, exists, forAll, iterate
- Alt sınıflar bunlar miras alır. Bunun dışında özelleşmiş işlemleri de olabilir.
- OCL referans manuelinde hepsini bulabilirsiniz.

Diğer OCL Özellikleri

Kısıtlar, gezinti ve koleksiyonlar dışında OCL'in daha az kullanılan özellikleri

Tuple: (Değişkenler grubu) Programlama dilllerindeki struct/record tarzı yapılar.

Enumaration: (Sayım) Java ve diğer dillerdekiler gibi.

Message Expression: (Mesaj ifadesi) bir işaretin gönderildiğinin belirtilmesi ya da bir işlemin tetiklenmesi

UML Metamodel Erişimi

Automatic Flattening: (otomatik düzleştirme): Gezintide, üzerinde pek çok bileşeni olan iki ilişki olsun (kümeler kümesi, kümeler yığını gibi).
- OCL'de iki seviyeyi de sözdizimimizde belirtmemize gerek yoktur.

OCL – Son sözler

Görece kolay bir dil

CASE araçları destek veriyor

Sistemimizin özelliklerini kesin olarak tanımlayabilmemize olanak sağlıyor

Yapısal ve davranışsal diyagramların tamamlayıcısıdır.

OCL ile istediğiniz kadar kesin tanım yapabilirsiniz.

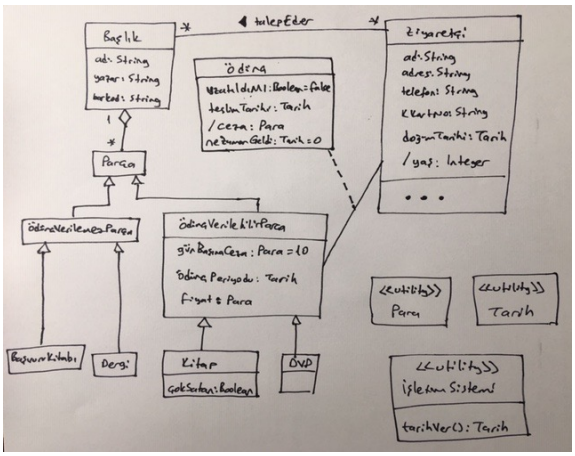
OCL – Kütüphane Örneğimiz

UML DİYAGRAMI İLE AÇIKLAYAMADIĞIMIZ KISIMLAR İÇİN OCL'İ KULLANALIM

Ali'nin Gereksinimleri

1. Her ziyaretçi sistemde olduğu sürece bir tekil kütüphane kartına sahiptir.
2. Kütüphane her ziyaretçi için en az ad, adres, telefon, kütüphane kartı numarasını bilmelidir.
3. Kütüphane herhangi bir anda bir ziyaretçinin ödünç aldığı parçaları, teslim zamanlarını, ödemediği gecikme cezalarını bilebilmeli veya hesaplayabilmelidir.
4. 12 yaş altındaki çocuklar için özel bir kısıtlama vardır. Bir kerede en fazla 5 parça ödünç alabilirler.
5. Ziyaretçi kitap veya DVD ödünç alabilir.
6. Kitaplar üç hafta için ödünç alınabilir. Çok satan kitaplar için limit iki haftadır.
7. DVD'ler iki hafta için ödünç alınabilir.
8. Gecikme cezası her parça için günlük on TL'dir. Gecikme cezası parçanın fiyatından daha yüksek olamaz.
9. Kütüphanede ödünç alınamayan başvuru kitapları ve dergiler de vardır.
10. Ziyaretçi, kütüphanede olmayan bir kitabı ya da DVD'yi talep edebilir.
11. Ziyaretçi bir parça için sadece bir kez süre uzatma yapabilir. Parça için çok talep varsa, parçayı geri getirmelidir.

Sınıf Modelimiz



Gereksinimlerimiz yeterince aydınlatıldı mı?

2. Kütüphane her ziyaretçi için en az ad, adres, telefon, kütüphane kartı numarasını bilmelidir.
- Gereksinimler bilgilerin varlığı gibi ise, bunlar diyagramda (özelliklerde) yeterince anlatılır.

3. Kütüphane herhangi bir anda bir ziyaretçinin ödünç aldığı parçaları, teslim zamanlarını, ödemediği gecikme cezalarını bilebilmeli veya hesaplayabilmelidir.
- Burada bilgiyi üretmek için sorgular yapılmalı, Bir işlem varsa, diyagramda bunun imzasını verebiliriz. Hesaplamayı OCL ile tanımlarız.

9. Kütüphanede ödünç alınamayan başvuru kitapları ve dergiler de vardır.
- Burada da bilgi var ve genelleme ile tüm bilgiyi sağladık.

5. Ziyaretçi kitap veya DVD ödünç alabilir.
- Burada da bilgi var ama nasıl ödünç alınamacağını anlatamıyoruz. Ödünç alma işleminin detaylarını vermedik. Örneğin çocuk ise limit var vs.

Sınırlama: Ya diyagramdaki tanım desteklenmeli, ya da yeni bir tanım getirilmelidir.

6. Kitaplar üç hafta için ödünç alınabilir. Çok satan kitaplar için limit iki haftadır.

Iki hafta – Üç hafta → kesin sayılar. Belirgin sayılar varsa, bir kontrol olmalıdır.

Bir kontrol varsa, bunu tanımlamalıyız.
- OCL ile diyagramda tanımladığımız elemanlara bu gibi gereksinimleri ekleyebiliriz.
- FOL ile güçleri denk olduğu için, sistemin işlevsel gereksinimlerini istenilen detaya kadar tanımlayabilmemizi sağlar.

İlk adım: diyagramdaki hangi nesne OCLifadesini yerleştirmek için uygundur?
- Context: OCL'i tanımlayacağımız sınıf ya da metodun adı
- Bu gereksinim için context: Kitap

İkinci adım: Kısıt sistemle mi, belirli bir sınıfla mı ya da belirli bir işlemle mi ilgili?
- Sistem veya sınıf ile ilgili ise invariant (sınıf/sistemle ilgili ne doğru olmalı),
- İşlemle ilgili ise ön ve son koşullar.
- Bu gereksinim için kısıt: invariant (diyagram gibi tek kesin bir cevabı yoktur. ödünçAl işlemine sonkoşul olarak da yazabilirdik. Limit hep olduğu için bizimki daha mantıklı)

Gereksinim 6: OCL

Kullanacağımız sınıfı ve kısıt tipini belirledikten sonra, kısıtı OCL ile yazabiliriz:

```
context Kitap inv:
      if cokSatan then
            oduncPeriyodu = 2 --hafta
      else
            oduncPeriyodu = 3
      endif
```

Koşullu ifade, programlama dillerindeki deyim gibi değildir, Durum değişikliği yerine, değer üretir. İki değer olasılığı var ve ikili ifadenin (cokSatan) değerine göre, oduncPeriyodu ya 2, ya 3'tür.

Gereksinim 6: OCL

Yazdığımız ifade tek bir değişmez (invariant) kısıt (sistemin bir özelliğinin hep alması gereken bir değer) tanımlar.

Burada 2 ve 3 rakamlarını açıkça kullandık tarih olduklarını söylemedik. Ancak diyagramdan Tarih tipinde olduklarını biliyoruz.

- Türlerin uyuşmasını sağlamalıyız.
- Tarih sınıfımız var, düzgün şekilde kullanmalıyız.

Her OCL kısıtı belirli bir sınıfın kapsamında yorumlanır.

- Noktasız gördüğümüz tüm ifadeler sınıfın kapsamındadır (özellikler, işlemler)
- Diyagramda başka kapsamdaki sınıfları açıkça isimleriyle kullanabiliriz.
-

Gereksinim 7:DVD'ler iki hafta için ödünç alınabilir.

```
context DVD
      inv: oduncPeriyodu = 2
```

Burada şartlı ifadeye de gerek yok.

Değişmezler, özelliklerin alacakları değerleri belirlemek için faydalıdır.

Gereksinim 3: Kütüphane herhangi bir anda bir ziyaretçinin ödünç aldığı parçaları, teslim zamanlarını, ödemediği gecikme cezalarını bilebilmeli veya hesaplayabilmelidir.

Burada sorgu işlemleri var. Hiçbir şeyi değiştirmeden özelliklerin değerlerini sorgulayan işlemler.

OCL'de işlemleri ilk ve son koşul kısıtları ile tanımlayabiliriz.

İlk kısmına odaklanalım. Ziyaretçinin ödünç aldığı parçalar nelerdir?

Diyagramda Ziyaretçi'de ödünçAldıkları davranışı eklemiştik.

- Bu davranışın geri dönüşü olarak sadece belirli ziyaretçinin ödünç aldığı parçaları listeleyebilmeliyiz.
- İşlem, ilişkiyi sorguluyor.

```
context Ziyaretci::oduncAldiklari() :
      Set(OduncVerilebilirParca)
```

Şu anda görünür argümanımız yok. Nesneye yönelik programlamada görünmez argüman var mıdır?

Ama geri dönüş değerimiz var. Sıralı olmak zorunda değil,

Bir kitabın pek çok kopyasını da ödünç alabiliriz. Ama bir kitabı aynı anda iki kere ödünç alamayız.

Gereksinim 3: Kütüphane herhangi bir anda bir ziyaretçinin ödünç aldığı parçaları, teslim zamanlarını, ödemediği gecikme cezalarını bilebilmeli veya hesaplayabilmelidir.

```
context Ziyaretci::oduncAldiklari() :
      Set(OduncVerilebilirParca)
```

Bir kitabı ödünç alsak, teslim zamanını geçirsek, borcumuz olsa, sonra geri getirsek, sistem bunu yine de hatırlayabilmeli. Kayıt kalmalı. Bu yüzden küme koleksiyonu uygun.

Sonra ön koşulları sorgularız.

- İşlemin çalışmasının anlam ifade ettiği durum nedir?
- Bu problemde yok. Durumda değişikliğe sebebiyet vermeden özelliklerin değerini sorgulayan işlemlerde ön koşul genellikle olmaz.
- Ya pre: True, ya da hiç yazmayız.

Gereksinim 3: Kütüphane herhangi bir anda bir ziyaretçinin ödünç aldığı parçaları, teslim zamanlarını, ödemediği gecikme cezalarını bilebilmeli veya hesaplayabilmelidir.

Sonraki adımda işlemden dönen değeri belirleriz.

Ödünç ilişkisi üzerinden ilgili (bu ziyaretçinin aldığı) ödünçAlınabilirParça'lara erişiriz.

- Hem ödünç alıp henüz getirmediklerimiz,
- Hem de ödünç alıp, zamanını geçirip, geç getirdiklerimiz

```
context Ziyaretci::oduncAldiklari() :
      Set(OduncVerilebilirParca)
post:
      result = Odunc.oduncVerilebilirParca
```

Gereksinim 4: 12 yaş altındaki çocuklar için özel bir kısıtlama vardır. Bir kerede en fazla 5 parça ödünç alabilirler.

Tüm ziyaretçiler için kısıt yok ve tüm çocuklar için kısıt var.

Değişmez mi yoksa önkoşul mu?
- Çocuktaki kontrol her ödünç alma işleminde yapılmalı!

```
context Ziyaretci::oduncAl(i: oduncVerilebilirParca)
      pre: yas <= 12 implies oduncAldiklari() -> size() < 5
```

Ön koşul doğru olmazsa işlem çalışmayacak.

Çocuk ise önkoşul var (FOL ==>)

Koşul: alınanlar kümesinin boyutunun 5'den küçük olması DİKKAT!! <= değil.

Bu kısıt çocukların kaç tane ödünç aldıklarını açıkça göstermez. Asla 5'ten fazla olmamasini garanti altına alır.

Post kısmı yok, şu anda tam bir tanım değil.

Gereksinim 5: Ziyaretçi kitap veya DVD ödünç alabilir.

Şimdiye kadar özelliklerin değerine bakan sorgulama işlemlerine baktık.

İşlemlerimiz, elemanlarımızın durumlarını da değiştirebilir.
- Veritabanında kayıt eklemek, güncellemek gibi işlemler yapmalıyız.

Her zaman aynı çıktıyı üreten işlemler metematiksel fonksiyonlar gibidir ve saf fonksiyonlardır.

Saf olmayan fonsiyonların yan etkileri vardır. Ör. VT'da değer değişikliği, I/O işlemi, ekrana çıktı üretmek, …

```
Context Ziyaretci::oduncAl(i: oduncVerilebilirParca)
```

Bir context'te birden fazla kısıt yazacaksak hepsini and'leyebiliriz ya da her birini ayrı ayrı tanımlayabiliriz.

Gereksinim 5: Ziyaretçi kitap veya DVD ödünç alabilir.

Ön koşullar:

- ÖdünçAlınabilirParça var olmalı → yeni işlem tanımlarız (ödünçAlınabilirParça'da)
- Ziyaretçi, istek listesindeki ilk kişi olmalı → yeni işlem tanımlarız (ödünçAlınabilirParça'da)
- Bir de çocuklar için olan ön koşulumuz vardı
- Adım adım, yeni işlemler ekleriz. Algoritmayı parçalamak gibi.

```
context Ziyaretci::oduncAl(i: oduncVerilebilirParca)
pre:
    i.mevcutMu() and
    i.baskasiDahaOnceIstemediMi() and
    (yas <=12 implies oduncAldiklari() -> size() < 5)
```

oduncAl Son koşulları-1

Genellikle ön koşullar daha kolaydır.

oduncAl işlemi sonrasında bilgi sistemimizde ne etkiler olacak?

1. oduncAl ilişkisinde yeni bir bağlantı (yeni bir nesne örneği yaratılacak) ve bize döndürülecek.

2. İstek listesini güncellememiz gerek (eğer biz talep etmiştiysek ve ödünç aldıysak) (almamız için listenin en başında olmamız gerekli)

```
post: exists(
 o: Odunc |
 o.oduncVerilebilirParca = i and
 o.teslimTarihi = os.getDate()+ o.oduncVerilebilirParca.oduncPeriyodu and
 Odunc = Odunc@pre -> including(o) )
```

oduncAl Son koşulları-1

```
post: exists(
  o: Odunc |
  o.oduncVerilebilirParca = i and
  o.teslimTarihi=os.getDate()+ o.oduncVerilebilirParca.oduncPeriyodu and
  Odunc = Odunc@pre -> including(o) )
```

Exists: "there must exist" ∃ → o bizim ilişkideki yeni oluşan bağlantımız. Ilişkinin iki tarafını da tanımlıyoruz:

- Ilk cümlecik: oduncAl işlemi tamamlanınca mutlaka ilgili oduncVerilebilirParca'sı oduncAl(i)'nin argümanı olan bir "o" örneği yaratılmalı
- İkinci cümlecik: teslim tarihi (önceki OCL'den) ayarlanmalı
- Üçüncü cümlecik: işlem bitince nesnemiz ilişki kümesine eklenmeli

oduncAl Son koşulları-2

2. İstek listesini güncellememiz gerek (eğer biz talep etmiştiysek ve ödünç aldıysak) (almamız için listenin en başında olmamız gerekli)

Let ifadesi ile mevcut parçanın başlığını b olarak kısaltıp kullanıyoruz:

```
context Ziyaretci::oduncAl(i: oduncVerilebilirParca)
post: let b : Baslik = i.baslik in
  if b.talepler.ziyaretci -> includes(self)
  then talepler=talepler@pre->reject(baslik=b and ziyaretci=self)
  else
    true
  endif
```

Eğer bu başlığa ait talebi yapan ziyaretçi biz isek, talep listesinden kendimizi çıkarıyoruz.

Implies ile de yapabilirdik.

Ziyaretçi'nin Yaşı OCL

Veri türetmek için de OCL'i kullanabiliriz.

Bu sefer contextimiz bir özellik.

```
context Ziyaretci::yas: Date
  derive:
    OperatingSystem.getDate() - doğumTarihi
```

Eksiklerimiz

1, 3, 8, 10, 11. gereksinimlere hiç bakmadık.

İşlevsel olmayan gereksinimlere hiç bakmadık.

Türettiğimiz işlemlere (ör. mevcutMu()) hiç eğilmedik.

iadeEt, cezaÖde, talepİptali işlemlerine hiç bakmadık.

Tarih, Para gibi sınıfları varsaydık.

Parçaların ve Ziyaretçilerin nasıl yaratılacaklarını çalışmadık.

Özetle,

Tek doğru yok.

Analiz safhasında yeni gereksinimler doğabilir.

Açık olmayan noktalar ve hatalar olabilir.

Müşteriyle tekrar tekrar temasa geçmek zorunda kalabiliriz.

OCL ile sıkı düşünüp bu tip noktaları minimuma indirebiliriz.

Davranış Modellemesi